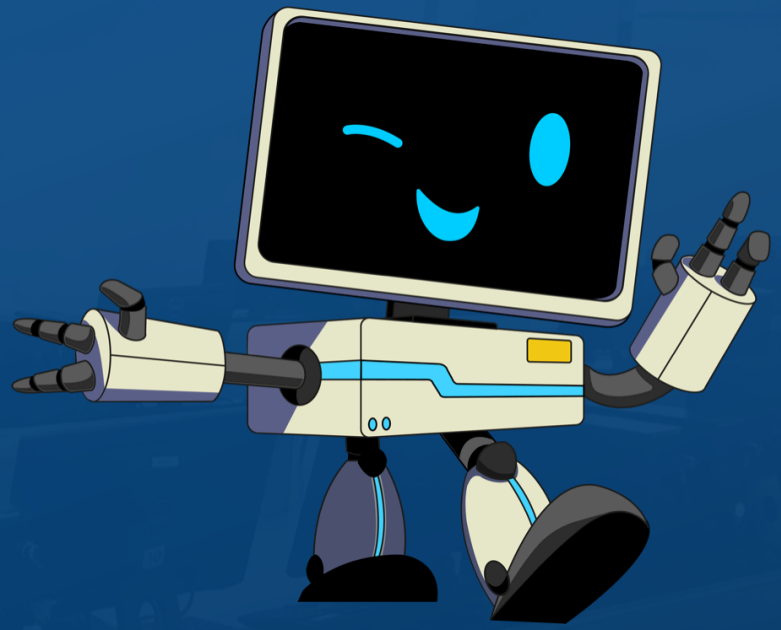
UIFCE
UNIDAD DE INFORMÁTICA



**Curso:**
Introducción a Python y R

**Sesión 4:**
Primer contacto con datos reales: pandas

INTRODUCCIÓN A PYTHON Y R · SESIÓN 4

# De la lista a la tabla


**Nombre del monitor(a): Yony Sebastián Chaparro Mesa**

Estudiante Auxiliar

***Correo:*** uniic_bog@unal.edu.co

***Tel:*** 3165000 ***Ext:*** 12301



*Facultad de Ciencias Económicas*
*Unidad de Informática*
*Sede Bogotá*

UNIVERSIDAD NACIONAL DE COLOMBIA

# Contenido

Sesión 4 · Python en Google Colab

1. **Repaso activo**
2. **¿Qué es un dataset?**
3. **Leer y explorar**
4. **Seleccionar y filtrar**
5. **Resumir y graficar**
6. **¡A practicar!**

1

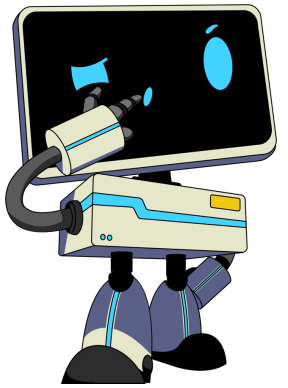
# Repaso activo

Lo que vimos en la sesión 3

**Correo:** uniic_bog@unal.edu.co
**Teléfono:** 3165000 **ext** 12301

UIFCE
UNIDAD DE INFORMÁTICA

# Calentamiento



**RETO**

Escribe una función `contar_mayores(valores, umbral)` que devuelva cuántos valores de una lista superan un umbral.

**Con `[3.5, 4.2, 2.8, 4.9]` y umbral `4.0`, ¿qué devuelve?**

```
def contar_mayores(valores, umbral):
    cuenta = 0
    for valor in valores:
        if valor > umbral:
            cuenta = cuenta + 1
    return cuenta

print(contar_mayores([3.5, 4.2, 2.8, 4.9], 4.0))   # 2
```

Devuelve **2**: solo 4.2 y 4.9 superan 4.0.

# 2

# ¿Qué es un dataset?

Datos organizados en una tabla

**Correo:** uniic_bog@unal.edu.co
**Teléfono:** 3165000 **ext** 12301

# De la lista a la tabla

**SESIÓN 3 · LISTA**

Muchos valores de **una sola cosa**.

```
irca = [1.7, 0.3, 5.0]
```

**SESIÓN 3 · DICCIONARIO**

Varias cosas de **un solo caso**.

```
fila = {"municipio": "Medellín", "irca": 1.7}
```

**HOY · TABLA**

Varias cosas de **muchos casos**.

| municipio | irca | nivel |
|---|---|---|
| Medellín | 1.7 | Sin riesgo |
| Abejorral | 0.3 | Sin riesgo |
| Abriaquí | 5.0 | Sin riesgo |

Un **dataset** es una tabla de datos. Con **pandas**, Python puede leerla y responder preguntas sobre ella.

# Filas y columnas

| departamento | municipio | irca | nivel |
|---|---|---|---|
| Antioquia | Medellín | 1.7 | Sin riesgo |
| Antioquia | Abejorral | 0.3 | Sin riesgo |
| Antioquia | Abriaquí | 5.0 | Sin riesgo |
| Antioquia | Alejandría | 0.2 | Sin riesgo |
| Antioquia | Amagá | 31.3 | Riesgo medio |

**Fila = observación**

Un caso. Aquí, **un municipio**.

**Columna = variable**

Lo que sabemos de cada caso. Aquí, su **IRCA**.

Cada columna tiene un solo tipo de dato: `irca` es un número; `nivel` es texto.

# ¿Qué es un archivo CSV?

Comma-Separated Values: valores separados por comas

## Por dentro: solo texto

Cada línea es una fila; las comas separan columnas.

```
departamento,municipio,irca,nivel
Antioquia,Medellín,1.7,Sin riesgo
Antioquia,Abejorral,0.3,Sin riesgo
Antioquia,Abriaquí,5,Sin riesgo
```

## En una hoja de cálculo

Los mismos datos, ordenados en celdas.

| departamento | municipio | irca | nivel |
|---|---|---|---|
| Antioquia | Medellín | 1.7 | Sin riesgo |
| Antioquia | Abejorral | 0.3 | Sin riesgo |
| Antioquia | Abriaquí | 5 | Sin riesgo |

La **primera línea** trae los nombres de las columnas. Casi cualquier programa puede abrir un CSV.

# Nuestro dataset: el agua que tomamos

## IRCA: Índice de Riesgo de la Calidad del Agua para consumo humano

Cada municipio tiene un puntaje de **0 a 100**. Entre más alto, mayor es el riesgo de enfermedades relacionadas con la calidad del agua.

| Sin riesgo | Riesgo bajo | Riesgo medio | Riesgo alto | Inviable sanitariamente |
|---|---|---|---|---|
| 0 a 5 | 5.1 a 14 | 14.1 a 35 | 35.1 a 80 | 80.1 a 100 |

**1.019**
municipios con dato

**8**
columnas

**2025**
año de los datos

Fuente: Instituto Nacional de Salud (SIVICAP), datos.gov.co. Licencia CC BY-SA 4.0. Versión adaptada para el curso.

# 3

# Leer y explorar

Lo primero que se le pregunta a un dataset

**Correo:** uniic_bog@unal.edu.co
**Teléfono:** 3165000 **ext** 12301

# Leer el archivo con pandas

```
import pandas (1) as pd (2)

url (3) = "https://raw.githubusercontent.com/…/agua_municipios_2025.csv"
df (4) = pd.read_csv(url) (5)
```

1. **pandas** es una librería para trabajar con tablas.
2. **pd** es su apodo: así la llamamos después.
3. **url** es una variable con la dirección del archivo.
4. **df** guarda la tabla: es un **DataFrame**.
5. **read_csv** lee el archivo y arma la tabla.

https://raw.githubusercontent.com/YonyChaparro/CL---Introduccion-a-python-y-R/main/datos/agua_municipios_2025.csv

# head(): las primeras filas

Sesion04.ipynb

```
df.head()
```

| | departamento | municipio | irca | nivel | irca_urbano | nivel_urbano | irca_rural | nivel_rural |
|---|---|---|---|---|---|---|---|---|
| 0 | Antioquia | Medellín | 1.7 | Sin riesgo | 0.8 | Sin riesgo | 4.8 | Sin riesgo |
| 1 | Antioquia | Abejorral | 0.3 | Sin riesgo | 0.3 | Sin riesgo | NaN | NaN |
| 2 | Antioquia | Abriaquí | 5.0 | Sin riesgo | 5.0 | Sin riesgo | NaN | NaN |
| 3 | Antioquia | Alejandría | 0.2 | Sin riesgo | 0.2 | Sin riesgo | NaN | NaN |
| 4 | Antioquia | Amagá | 31.3 | Riesgo medio | 0.6 | Sin riesgo | 55.9 | Riesgo alto |

- Las filas se numeran desde **0**, igual que las listas.
- **NaN** significa **dato faltante**. Y `df.head(10)` muestra 10 filas.

# shape y columns: el tamaño y los nombres

## `df.shape`

Cuántas filas y columnas hay.

```
df.shape
# Salida
(1019, 8)   # (filas, columnas)
```

## `df.columns`

Cómo se llama cada columna.

```
df.columns
# Salida
Index(['departamento', 'municipio', 'irca', 'nivel', 'irca_urbano',
       'nivel_urbano', 'irca_rural', 'nivel_rural'],
      dtype='str')
```

**¿Con o sin paréntesis?** `shape` y `columns` son datos que la tabla ya tiene: van **sin** paréntesis. `head()` e `info()` son acciones: van **con** paréntesis.

# info(): tipos y datos faltantes

```
df.info()
# Salida
RangeIndex: 1019 entries, 0 to 1018
Data columns (total 8 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   departamento   1019 non-null   str
 1   municipio      1019 non-null   str
 2   irca           1019 non-null   float64
 3   nivel          1019 non-null   str
 4   irca_urbano    1010 non-null   float64
 5   nivel_urbano   1010 non-null   str
 6   irca_rural     432 non-null    float64
 7   nivel_rural    432 non-null    str
```

- **Non-Null Count**: cuántos valores no faltan en cada columna.
- **float64**: número decimal. **str**: texto.
- A `irca_rural` le faltan **587** de 1.019 valores.

**Antes de confiar en una columna, revisa cuánto le falta.**

# Práctica en parejas

Exploren el dataset con `df.shape`, `df.head()` y `df.info()`

### ¿Cuántos municipios tiene el dataset?

**1.019**: es el primer número de `df.shape`.

### ¿Cuál es el quinto municipio de la tabla?

**Amagá**: tiene el índice 4, porque se cuenta desde 0.

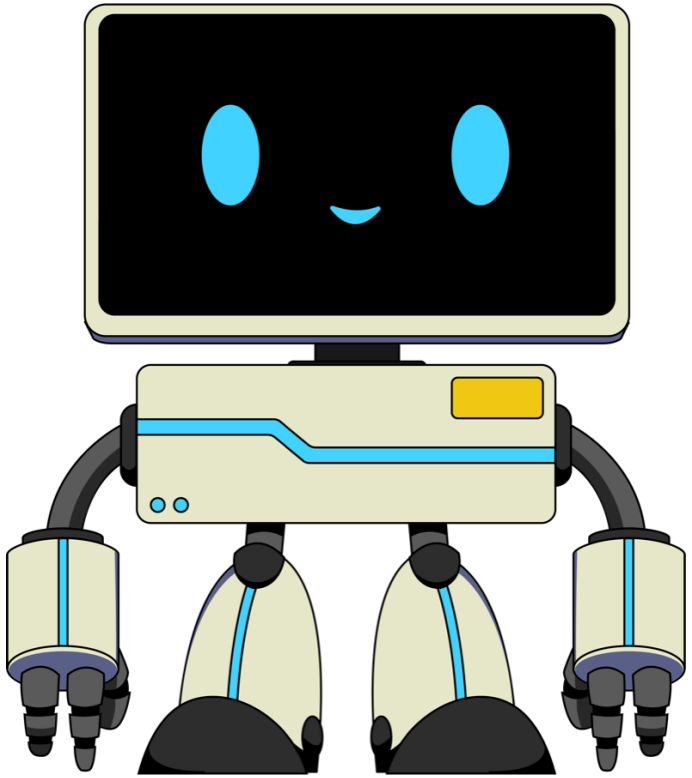
### ¿A cuántos municipios les falta el IRCA urbano?

**9**: `info()` muestra 1010 valores de 1019.

# Pausa

Estírate y toma agua.

# 10:00



# 4

# Seleccionar y filtrar

Quedarnos solo con lo que necesitamos

**Correo:** uniic_bog@unal.edu.co
**Teléfono:** 3165000 **ext** 12301

# Seleccionar columnas

## Una columna: `[ ]`

El nombre, entre comillas y corchetes.

```
df["municipio"].head(3)
# Salida
0     Medellín
1    Abejorral
2     Abriaquí
```

## Varias columnas: `[[ ]]`

Una **lista** de nombres dentro de los corchetes.

```
df[["municipio", "irca"]].head(3)
# Salida
   municipio  irca
0   Medellín   1.7
1  Abejorral   0.3
2   Abriaquí   5.0
```

**Ojo:** con un solo par de corchetes, `df["municipio", "irca"]` produce `KeyError`. Para pedir varias columnas se necesitan los dos pares.

# Filtrar filas: una condición por fila

```
df[df["irca"] > 35 (1)] (2)
```

(1) La **condición** se revisa en cada fila: da `True` o `False`.

(2) **df[ ]** conserva solo las filas con `True`.

```
altos = df[df["irca"] > 35]
len(altos)   # 64
```

CINCO FILAS DE EJEMPLO

| | municipio | irca | irca > 35 |
|---|---|---|---|
| 0 | Medellín | 1.7 | False |
| 4 | Amagá | 31.3 | False |
| 36 | Cisneros | 46.5 | True |
| 109 | Titiribí | 49.4 | True |
| 113 | Urrao | 97.0 | True |

Quedan **64 de 1.019** municipios: los que tienen riesgo alto o agua inviable sanitariamente.

# Es un for + if en una sola línea

## Con lo de la sesión 3

Recorrer los valores uno por uno.

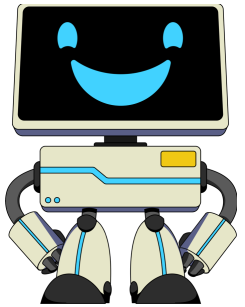
```
cuenta = 0
for valor in df["irca"]:
    if valor > 35:
        cuenta = cuenta + 1
print(cuenta)   # 64
```

## Con pandas

La misma pregunta, la misma respuesta.

```
print(len(df[df["irca"] > 35]))   # 64
```

Filtrar es aplicar una **condición** (sesión 2) a cada fila, sin escribir el **bucle** (sesión 3).



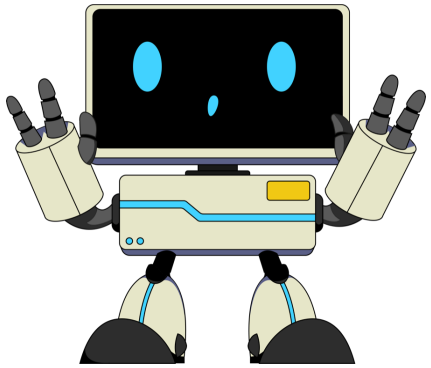
# Filtrar por texto

Con `==` para comparar, igual que en la sesión 2

```
cundi = df[df["departamento"] == "Cundinamarca"]
len(cundi)   # 113

cundi[["municipio", "irca", "nivel"]].head(3)
# Salida
        municipio  irca        nivel
442  Agua de Dios   3.3   Sin riesgo
443         Albán   6.0  Riesgo bajo
444      Anapoima   1.9   Sin riesgo
```

**El texto debe ser idéntico. Con “cundinamarca” quedan 0 filas, y no sale ningún error.**



# 5

# Resumir y graficar

De mil filas a una respuesta

**Correo:** uniic_bog@unal.edu.co
**Teléfono:** 3165000 **ext** 12301

UIFCE
UNIDAD DE INFORMÁTICA

# mean(): el promedio de una columna

| CÓDIGO | RESULTADO |
|---|---|
| `df["irca"].mean()` | 10.50 |
| `cundi["irca"].mean()` | 6.83 |
| `df["irca_urbano"].mean()` | 8.64 |
| `df["irca_rural"].mean()` | 21.83 |
| `df["irca"].max()` | 97.0 |

### ¿Qué se ve?

En promedio, el IRCA rural es más alto que el urbano.

### Con cuidado

Es un promedio entre municipios, no entre personas: cada municipio pesa lo mismo.

`mean()` ignora los **NaN**: el promedio rural usa solo los 432 municipios con ese dato.

# value_counts(): contar categorías

¿Cuántos municipios hay en cada nivel de riesgo?

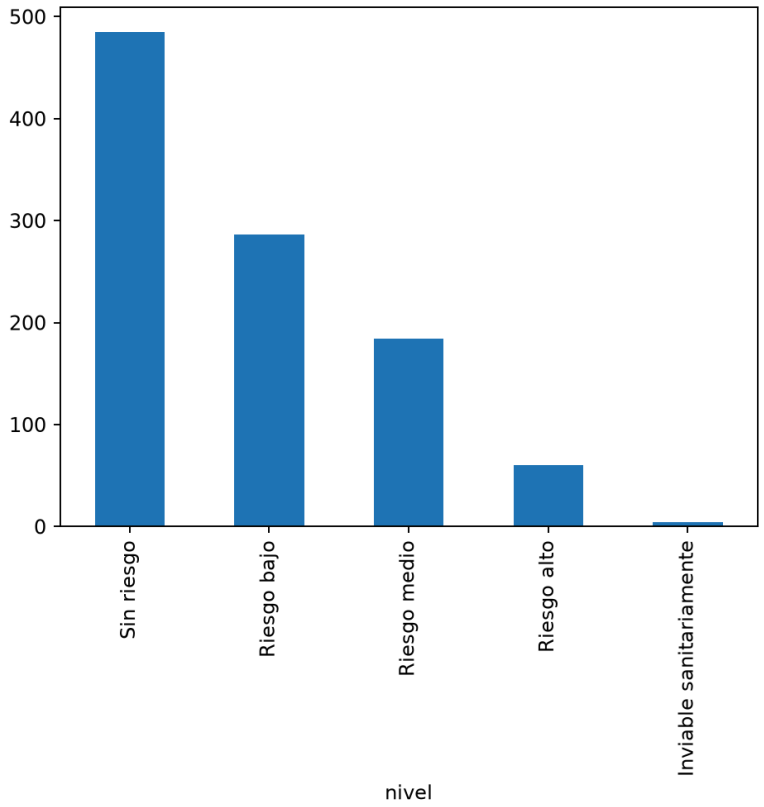
```
df["nivel"].value_counts()
# Salida
nivel
Sin riesgo                 485
Riesgo bajo                286
Riesgo medio               184
Riesgo alto                 60
Inviable sanitariamente      4
Name: count, dtype: int64
```

**DILO EN UNA FRASE**

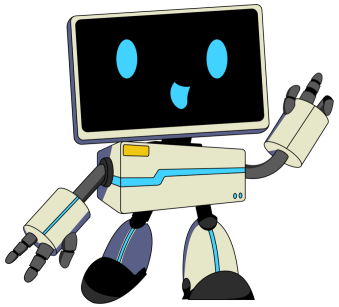
Casi la mitad de los municipios (**485 de 1.019**) está sin riesgo, pero **64** están en riesgo alto o tienen agua inviable sanitariamente.

# Una gráfica en una línea

```
df["nivel"].value_counts().plot(kind="bar")
```




- `value_counts()` cuenta y `.plot()` dibuja ese resultado.
- Cada barra es una categoría; su altura, cuántos municipios tiene.
- A simple vista: pocos municipios en los dos niveles más altos.



**Correo:** uniic_bog@unal.edu.co
**Teléfono:** 3165000 **ext** 12301

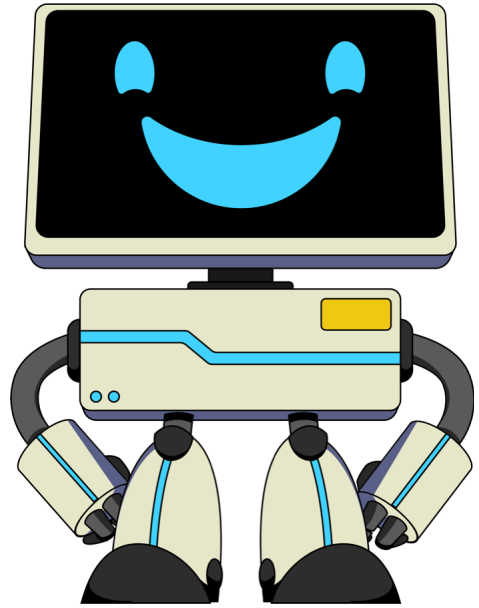


UIFCE
UNIDAD DE INFORMÁTICA

# Todo el recorrido en pocas líneas

**Leer** → **Explorar** → **Seleccionar** → **Filtrar** → **Resumir** → **Graficar**

```
import pandas as pd
df = pd.read_csv(url)                          # leer
df.head()                                      # explorar
df[["municipio", "irca"]]                      # seleccionar
df[df["irca"] > 35]                            # filtrar
df["irca"].mean()                              # resumir
df["nivel"].value_counts().plot(kind="bar")    # graficar
```



# 6

# ¡A practicar!

Sus propias respuestas con datos reales

**Correo:** uniic_bog@unal.edu.co
**Teléfono:** 3165000 **ext** 12301

UIFCE
UNIDAD DE INFORMÁTICA

# Errores frecuentes de hoy

### KeyError

```
df["Irca"]
KeyError: 'Irca'
```

Esa columna no existe: el nombre debe ser idéntico, con minúsculas y mayúsculas.

```
df["irca"]
```

### TypeError

```
df.shape()
TypeError: 'tuple' object
is not callable
```

**shape** es un dato de la tabla, no una acción: va sin paréntesis.

```
df.shape
```

### SyntaxError

```
df[df["nivel"] = "Riesgo alto"]
SyntaxError: ... Maybe you
meant '==' instead of '='?
```

Un solo = asigna. Para comparar se usan dos.

```
df[df["nivel"] == "Riesgo alto"]
```

# Ejercicios

## Avanza hasta donde llegues

### Básico — TODOS

1. ¿Cuántas filas y columnas tiene el dataset?
2. Muestra las primeras 10 filas.
3. Calcula el promedio de `irca_urbano`.

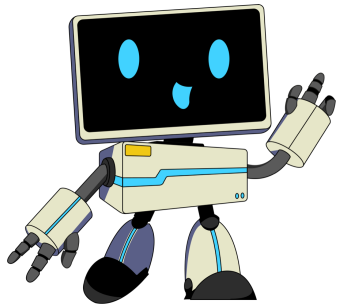
### Intermedio

4. Filtra los municipios con `irca` menor o igual a 5 y cuenta cuántos son.
5. Usa `value_counts()` en `departamento` e interpreta el resultado en una frase.

### Reto

6. Filtra Boyacá con `irca` mayor que 5 y grafica su `nivel`. Pista: `df[(cond1) & (cond2)]`.
7. Formula tu propia pregunta y respóndela con código.

**¿Atascado? Revisa los nombres con df.columns.**



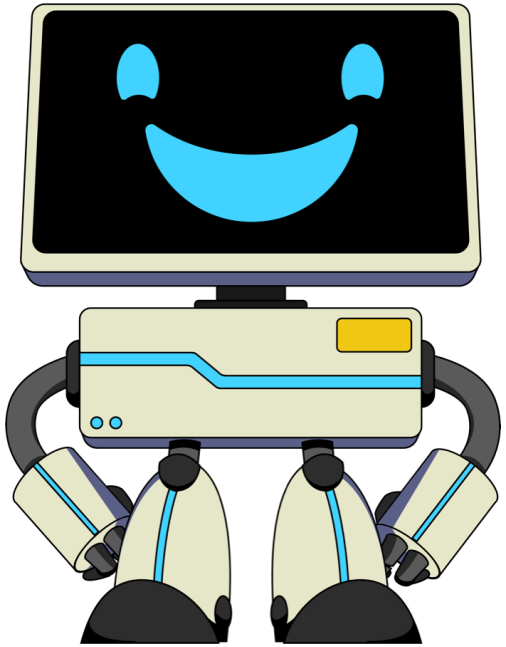
**Correo:** uniic_bog@unal.edu.co
**Teléfono:** 3165000 **ext** 12301



UIFCE
UNIDAD DE INFORMÁTICA

# Tres ideas clave de hoy

1. Un **dataset** es una tabla: las filas son casos y las columnas, variables.
2. Con pocas líneas de **pandas** se pueden leer, filtrar y resumir datos reales.
3. **Filtrar** es aplicar una condición a cada fila.



**Tarea corta**

Escribe 3 preguntas que quieras responder con este dataset.

**Obligatorio**

Crea tu cuenta gratis en **Posit Cloud** y verifica que abres un proyecto.

**Nivelación**

Hoja resumen con los comandos de pandas de hoy.

UIFCE
UNIDAD DE INFORMÁTICA

# ¡Gracias!

Próxima sesión: **El puente a R**. Lo mismo, en otro idioma.

***Correo:*** uniic_bog@unal.edu.co

***Tel:*** 3165000 ***Ext:*** 12301

